

Adaptive Smart Assessment and Integrity Detection System (ProctorLess)

Final Version

Course Title: Web Application and Internet

Course ID: CSE309

Section: 01

Semester: Autumn 2025

Submitted To: Md. Abu Sayed

Submitted By:

Name	ID
Taif Ur Rahman	**2210280**
Md. Jamiul Islam Nahin	**2210514**

Submission Date: December 13, 2025

Table of Contents

The Project SRS Approval Letter 2

1. Introduction 3

1.1 Purpose 3

1.2 Scope of Work (Project Overview) 3

1.3 Intended Audience 3

1.4 Definitions, Acronyms, and Abbreviations 3

2. General Description 4

2.1 Product Perspective and System Architecture 4

2.2 User Characteristics 4

2.3 Data Collection and Framework 4

2.4 Survey/Validation 4

3. Functional Requirements 5

3.1 Authentication and Roles 5

3.2 Course Management 5

3.3 Quiz Management 5

3.4 Student Quiz Engine 5

3.5 Integrity Monitoring 6

3.6 Results and Grading 6

3.7 Administration 6

3.8 API Integration 6

4. Interface Requirements 7

4.1 User Interface (HCI Layer) 7

4.2 Software Interfaces 7

4.3 Hardware 7

5. Non-Functional Attributes 7

5.1 Security 7

5.2 Performance and Reliability 8

5.3 Privacy and Compliance 8

6. Risk Management 8

6.1 Purpose and Scope 8

6.2 Major Identified Risk Factors 8

7. Change Management Process 9

7.1 Goal and Responsibility 9

7.2 Change Management Process Steps 9

7.3 Documentation and Control 9

8. Preliminary Schedule and Budget 9

9. Conclusion 9

Appendices 10

A. Glossary of Terms 10

B. Document Revision History 10

C. Change Log 10

Uses of SRS Document 11

FAQs on SRS Format 11

The Project SRS Approval Letter

I ……. understand that this SRS contains all the Software Requirement Analysis that has been precisely documented for understanding how the ProctorLess Platform will operate in the real world. I also understand that by approving this SRS, I will not be able to add further requirements to the ProctorLess Platform. I am hereby providing my authorization to proceed with the design and development phase.

Name: _____

Designation: _____

Signature: _____

Date: _____

1. Introduction

The Introduction section sets the stage by defining the document's purpose, scope, audience, and key terms, ensuring a unified understanding among stakeholders.

1.1 Purpose

The primary purpose of this SRS is to define complete, clear, and testable requirements for ProctorLess, an adaptive online assessment system. It aims to gain agreement with stakeholders regarding the system's functionality, which centers on role-based dashboards, real-time integrity monitoring, and AI-assisted capabilities.

1.2 Scope of Work (Project Overview)

The project involves the development of a web application designed for creating and taking quizzes within a structured course environment. It features role-based administration, providing distinct dashboards for students, professors, and administrators. A key component is the real-time integrity monitoring system, which utilizes a privacy-first browser extension to detect tab switching without invasive surveillance. Additionally, the system integrates AI capabilities to assist in generating quiz variations and automating the grading process, thereby streamlining the assessment workflow.

1.3 Intended Audience

This document is intended for the development team, project managers, QA engineers, and academic stakeholders. It is organized to guide implementation and testing, serving as the primary reference for system behavior.

1.4 Definitions, Acronyms, and Abbreviations

The document uses standard software engineering terminology alongside project-specific terms. SRS refers to Software Requirements Specification. HCI stands for Human-Computer Interaction. Supabase is the Backend-as-a-Service provider used for database, authentication, and realtime features. AI refers to the Artificial Intelligence integrations powered by Google Gemini via Edge Functions.

2. General Description

2.1 Product Perspective and System Architecture

The system relies on a modern web architecture using Next.js 16 (App Router) for the frontend and React 19 for the user interface components. The backend infrastructure is managed by Supabase, which handles Authentication, Database, Storage, and Realtime subscriptions. A dedicated Chrome browser extension serves as the integrity monitoring agent, communicating tab-switch events directly to the backend. AI features are decoupled into Supabase Edge Functions that securely interact with Gemini APIs.

2.2 User Characteristics

The system supports three distinct user classes. Students require a clear, distraction-free interface for taking timed quizzes with assurance that their progress is autosaved. Professors need efficient workflows for creating courses, managing question banks, and

reviewing AI-assisted grades. Administrators require high-level oversight tools to manage user roles and audit system health.

2.3 Data Collection and Framework

Data collection focuses on academic assets and integrity logs. Question banks and grading artifacts are stored in designated storage buckets, while relational data such as user profiles, course associations, and quiz submissions are maintained in the Postgres database. Integrity data is strictly limited to timestamped events of tab switching, with specific exclusions for 'allowed websites' to prevent false positives.

2.4 Survey/Validation:

To ensure the system addresses real-world academic integrity challenges, requirements were validated through competitive analysis of existing proctoring tools and feedback from academic stakeholders. The analysis highlighted a critical need for non-invasive monitoring (privacy-first) and faster grading workflows, directly influencing the inclusion of the 'Integrity Monitor' extension and AI grading features.

3. Functional Requirements

These sections define the specific activities the system must perform.

3.1 Authentication and Roles

The system shall handle user authentication securely via Supabase Auth. Upon registration or assignment, the system shall associate each user with a single specific role: student, professor, or admin. Immediately following a successful login, the system shall strictly route the user to their role-appropriate dashboard, ensuring no unauthorized access to administrative or professorial views.

3.2 Course Management

Professors shall have the ability to create new courses by providing a title and description. To facilitate enrollment, the system shall generate unique join codes for every course. Students shall be able to join these courses by entering a valid code, after which the system will grant them access to the course content.

3.3 Quiz Management

The system shall enable professors to create quizzes linked to specific courses and configure integrity settings, including the definition of allowed websites. Professors shall upload question banks, which the system shall parse using OCR or AI extraction techniques. Furthermore, the system shall support the generation of quiz variations from a master question set to prevent cheating.

3.4 Student Quiz Engine

During an assessment, the system shall present questions in a randomized order from the configured bank. To prevent data loss, the system shall autosave answers periodically and whenever a change is detected. The system shall enforce all timing constraints and automatically submit the quiz when the time expires, persisting the submission with a unique identifier.

3.5 Integrity Monitoring

The integrity monitoring relies on a browser extension that shall detect tab switches whenever the monitoring mode is armed. The system shall record these events with timestamps and referrer context but shall exclude any websites explicitly listed in the 'allowed' configuration. These integrity events shall be displayed on the results dashboard for professor review.

3.6 Results and Grading

The system shall display a list of submissions and grades for every quiz. Professors shall have the option to invoke AI-assisted grading via a Supabase Edge Function, which provides preliminary scores and feedback. Professors shall retain the ability to override these grades manually and save the final results. Summary statistics for both grades and integrity violations shall be provided.

3.7 Administration

Admins shall have exclusive access to manage the user base, including the ability to create, delete, and assign roles to user accounts. The admin dashboard shall provide overview statistics of total users, quizzes, and submissions, along with the capability to browse and audit lists of courses and quizzes.

3.8 API Integration

The system shall provide secure REST endpoints for AI operations, including extracting questions from files and generating quiz variations. All payloads sent to these endpoints shall be validated, and responses shall be structured in a normalized format for frontend consumption.

4. Interface Requirements

4.1 User Interface (HCI Layer)

The user interface shall adhere to a minimal, academic design philosophy with high-contrast text and responsive layouts. Navigation shall be strictly role-based, ensuring users only see options relevant to their tasks. Administrative interfaces shall use tabbed layouts to organize complex data tables efficiently.

4.2 Software Interfaces

The system interfaces with Supabase for all backend operations, including Auth, Database, and Realtime features. External AI services are accessed via secure API endpoints (e.g., /api/gemini/extract). Data exchange occurs via JSON over HTTPS, with assets stored in specific Storage buckets.

4.3 Hardware:

Since the application is web-based, it requires no specialized hardware. Users (students and professors) only require a standard computer (Desktop or Laptop) with an active internet connection (Wi-Fi or Ethernet) and a modern web browser to access the system.

5. Non-Functional Attributes

5.1 Security

The system shall implement robust security measures, including role-based authorization for all protected routes. Secrets and API keys shall be stored server-side and never exposed to the client. Row Level Security (RLS) policies shall be applied to the database to ensure data isolation.

5.2 Performance and Reliability

The system shall ensure that 95% of dashboard loads complete within 2 seconds on standard broadband connections. Quiz autosaving mechanisms must complete within 1 second to ensure a seamless student experience. Realtime subscriptions shall automatically reconnect in the event of transient network failures.

5.3 Privacy and Compliance

To maintain student privacy, the integrity monitoring extension shall only log data during active quiz sessions and must strictly exclude allowed domains from violation logs. No webcam or biometric data shall be collected.

6. Risk Management

This section describes the strategies and methods used to identify and avoid potential risks throughout the life cycle of the "ProctorLess" project .

6.1 Purpose and Scope

Risk management must identify all kinds of risks associated with the project. The goal is to monitor the project closely to prevent common risk factors that historically cause project failure . Risk factors are associated with three main areas:

1. **Nature of the work:** The scope of the solution and the target environment.
2. **Project and Team Environment:** Internal dynamics and resources.
3. **Technical Aspects:** Risks involved with the system technology. For ProctorLess, technical risks relate heavily to the integration of the **Gemini AI API** and the **Chrome Extension architecture** .

6.2 Major Identified Risk Factors

The following common risk factors must be closely monitored:

1. **Personnel Shortfalls:** Inadequate staffing or loss of key personnel.
2. **Unrealistic Schedules:** Overly aggressive planning or deadlines.
3. **Developing the Wrong User Interface:** A critical risk for this project, as the academic dashboard must be minimal and high-contrast for clarity.
4. **Gold Plating:** Adding more functionality or features than necessary (e.g., adding complex biometric tracking beyond simple tab monitoring).
5. **Continuing Stream of Requirements Changes:** Frequent, unmanaged changes that destabilize the scope.
6. **Real-Time Performance Shortfalls:** Failure to meet performance expectations (e.g., the browser extension must log events instantly without lagging the student's browser).

7. Change Management Process

The requirement change management process defines the set of activities that need to be performed when there are new requirements or changes to existing requirements.

7.1 Goal and Responsibility Goal:

The basic goal is to control requirement changes and minimize the impact of changes on the project. The process tries to ensure that a project succeeds despite requirement changes . **Responsibility:** The project team leader is primarily responsible for executing the process to incorporate the change in the project.

7.2 Change Management Process Steps

The process requires the execution of the following major steps when a requirement change request is received:

1. **Log Changes:** A change request log will be maintained.
2. **Perform Impact Analysis:** Analyze the change on the work products (DB schema, API endpoints).
3. **Estimate Effort:** Estimate the effort needed for this change request.
4. **Re-estimate Delivery Schedule**.
5. **Perform Cumulative Cost Impact Analysis**.
6. **Review Impact** with stakeholders if thresholds are exceeded.
7. **Obtain Sign-Off**.
8. **Rework Work Products**.

7.3 Documentation and Control

1. **Change Log:** The change request log will track each request, including the request number, description, impact, and status.
2. **SRS Modification:** The change request is incorporated into the requirement specification document, usually as appendices.
3. **Monitoring:** Monitoring of approved change requests ensures proper implementation and allows the team to deliver the ProctorLess application in a timely manner .

8. Preliminary Schedule and Budget

The project schedule is derived from the technical design phases, covering Authentication, Course Management, Quiz Engine, Integrity Monitoring, and AI Integration. The budget relies on free-tier services (Supabase, Vercel), with potential costs allocated for AI API usage, managed via strict quota limits.

9. Conclusion

This SRS outlines the design and requirements for ProctorLess, a secure and adaptive assessment platform. By leveraging Next.js for the interface and Supabase for backend reliability, coupled with AI-driven grading, the system aims to modernize academic testing while maintaining integrity. This document serves as the primary reference for the development and testing phases.

Appendices

A. Glossary of Terms

- **SRS:** Software Requirements Specification – A document that describes what the software will do and how it will be expected to perform.
- **HCI:** Human-Computer Interaction – The design and implementation of the user interface, focusing on user experience.
- **Supabase:** The Backend-as-a-Service (BaaS) provider used for the project's database, authentication, and realtime features.
- **Integrity Event:** A specific logged instance where the browser extension detects a user switching tabs away from the active quiz.
- **Edge Function:** Serverless functions (hosted on Supabase) used to securely call external AI APIs like Gemini.
- **Next.js:** The React framework used for building the frontend of the application.

B. Document Revision History

A log detailing the version number, date of release, and activity for each revision

Version	Date	Description	Author
v1.0	December 13, 2025	Initial Final Release	Taif Ur Rahman

C. Change Log

A log maintained to keep track of all change requests.

Change ID	Description	Impact Analysis	Status
CL-001	Consolidated admin panel into a single /dashboard view instead of separate dropdown links.	Reduced navigation complexity; required updating routing logic.	Approved
CL-002	Integrated OCR/Gemini parsing for question bank uploads.	Added dependency on external AI API; required error handling for timeouts.	Approved

CL-003	Added "Allowed Websites" whitelist to Integrity Monitor.	Reduced false positives in integrity logs; required schema update for Quiz settings.	Approved

Uses of SRS Document

The SRS serves multiple critical functions for the project team and stakeholders. It acts as a complete and precise documentation for understanding exactly how the system will operate in the real world. It describes the functional and non-functional requirements of the proposed system in substantial detail.

It defines how the application will interact with system hardware, other programs, and human users in a wide variety of real-world situations. The SRS minimizes the time and effort required to achieve desired goals and helps minimize development costs. Once approved, it guides the project personnel in understanding the project and management structure more clearly.

FAQs on SRS Format

The SRS format is standardized to ensure clarity and comprehensiveness.

Q1: What is the primary content of an SRS? Ans: An SRS is fundamentally a description of the software system to be developed. It clearly lays out the functional requirements and the non-functional requirements (or software attributes).

Q2: Does the SRS detail implementation specifics? Ans: The SRS focuses on *what* the software must do and how it will perform. While it outlines the development technology and architectural approach (like Next.js or Supabase), the detailed design and implementation specifics are often documented elsewhere, or in the Specific Requirements section.

Q3: Why are non-functional attributes important? Ans: Non-functional attributes (like security, response time, scalability, portability) define the quality and constraints of the system. These parameters are crucial for ensuring the system is robust and meets real-world expectations beyond just its core features.